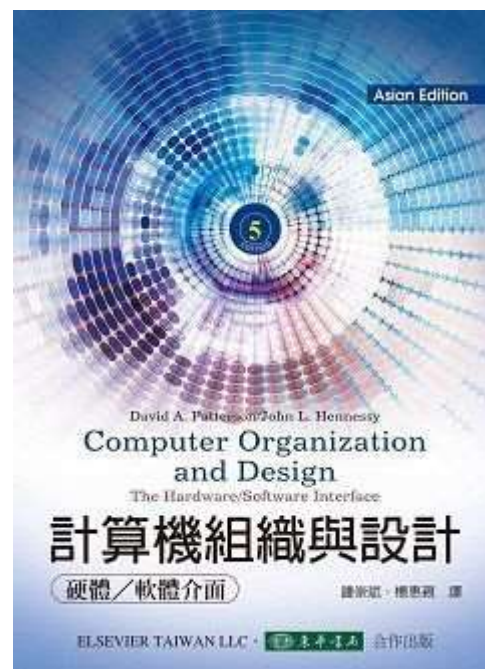


教科書



計算機組織與設計: 硬體/軟體的介面 5/e
Patterson (Asian Edition)
作者: 鍾崇斌、楊惠親 譯; **David A. Patterson**

**Computer Organization and Design 5/e
(Asian Edition)**
作者: **David A. Patterson, John L. Hennessy**

第 1 章

計算機抽象化與科技

- 1.1 介紹
- 1.2 計算機架構中的八大理念
- 1.3 你的程式之下
- 1.4 覆蓋之下
- 1.5 建構處理器與記憶體的技术
- 1.6 效能
- 1.7 功耗障壁
- 1.8 巨變：由單處理器轉移至多
處理器
- 1.9 實例：測試Intel core i7
- 1.10 謬誤與陷阱

1.1 介紹



- 計算機在汽車中：1980
- 手機：1980
- 人類基因計畫：1990
- 全球網路：1990
- 搜尋引擎：1990

計算應用的種類與它們的特性

大體而言，計算機被運用在三個不同種類的應用：

- 個人型計算機(Personal computers, PCs)
 - 強調以低成本提供單一使用者不錯的效能並能執行其他公司的軟體
- 伺服器(servers)是昔日大型主機、迷你計算機及超級電腦的現代版，通常經由網路來使用
 - 提供計算及輸出入容量上更大的擴充性
 - 也強調可靠度
 - Range from small servers to building sized

計算應用的種類與它們的特性

- 嵌入式計算機(embedded computers)在應用與效能的範圍最廣
 - 設計來執行單一應用或一組相關的應用，該等應用一般與硬體整合在一起，以單一系統型態呈現
 - 大部分使用者從不知道他們正在使用這類計算機
 - 通常有其獨特的應用需求以及最低效能和嚴格的成本與功耗限制
 - 通常更不能容忍失效
 - 經由簡單化或冗餘(redundancy)技術來達成

進入後個人電腦時代

- ▣ 個人行動裝置(personal mobile device, PMD)由電池驅動、無線聯線至網際網路且一般售價為數百美元
 - 和個人電腦一樣，使用者可於其上執行可下載的軟體(“apps”)
 - 可能使用觸控螢幕甚或語言輸入
- ▣ 庫房規模計算機(Warehouse Scale Computers, WSCs)與雲端計算(Cloud Computing)
 - 軟體寫作者常常會將他們的應用程式一部分執行於個人行動裝置上而一部分執行於雲端

What is a WSC

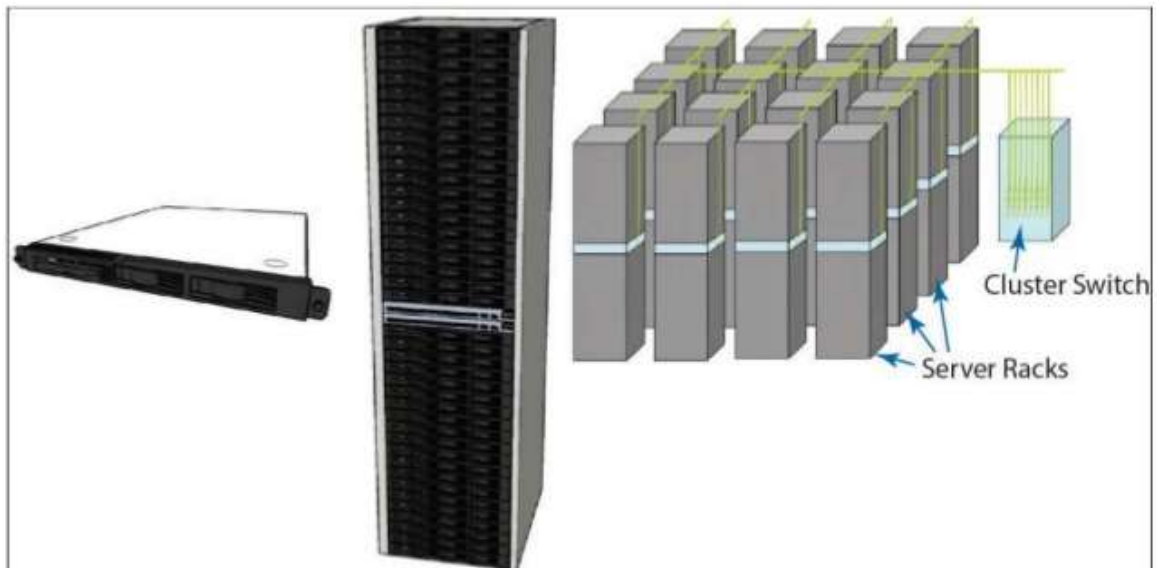


Warehouse-Scale Computer

- Scalable
- Distributed
- Cost efficiency



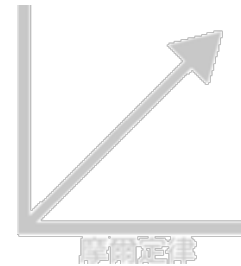
ARCHITECTURAL OVERVIEW OF WSCS



1.2 計算機架構中的八大理念

1. 配合摩爾定律作設計

- 該定律指出IC 容量於每18 至24 個月即加倍



2. 用抽象化來簡化設計

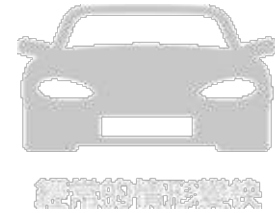
- 一個硬體與軟體的抽象化 (abstraction) 來代表在不同層次中的設計
- 在電腦科學中，**抽象化**是將資料與程式，以它的語意來呈現出它的外觀，但是隱藏起它的實作細節。一個電腦系統可以分割成幾個抽象層 (Abstraction layer)，使得程式設計師可以將它們分開處理。



1.2 計算機架構中的八大理念

3. 使經常的情形變快

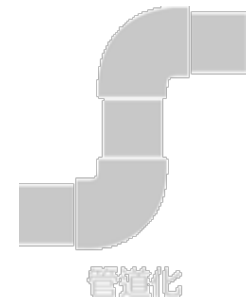
(讓 Clock 變快)



4. 經由平行性提升效能



5. 經由管道處理(pipelining)提升效能



1.2 計算機架構中的八大理念

6. 經由預測(prediction)提升效能
(分支預測)

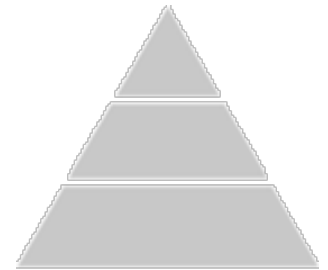


預測

7. 記憶體的分層(hierarchy of memories)

- 程式設計師希望記憶體要容量大、速度快且價廉

(SSD → DRAM → L3 → L2 → L1)
CPU 內 Cache



階層

8. 經由冗餘(redundancy)提升可靠性



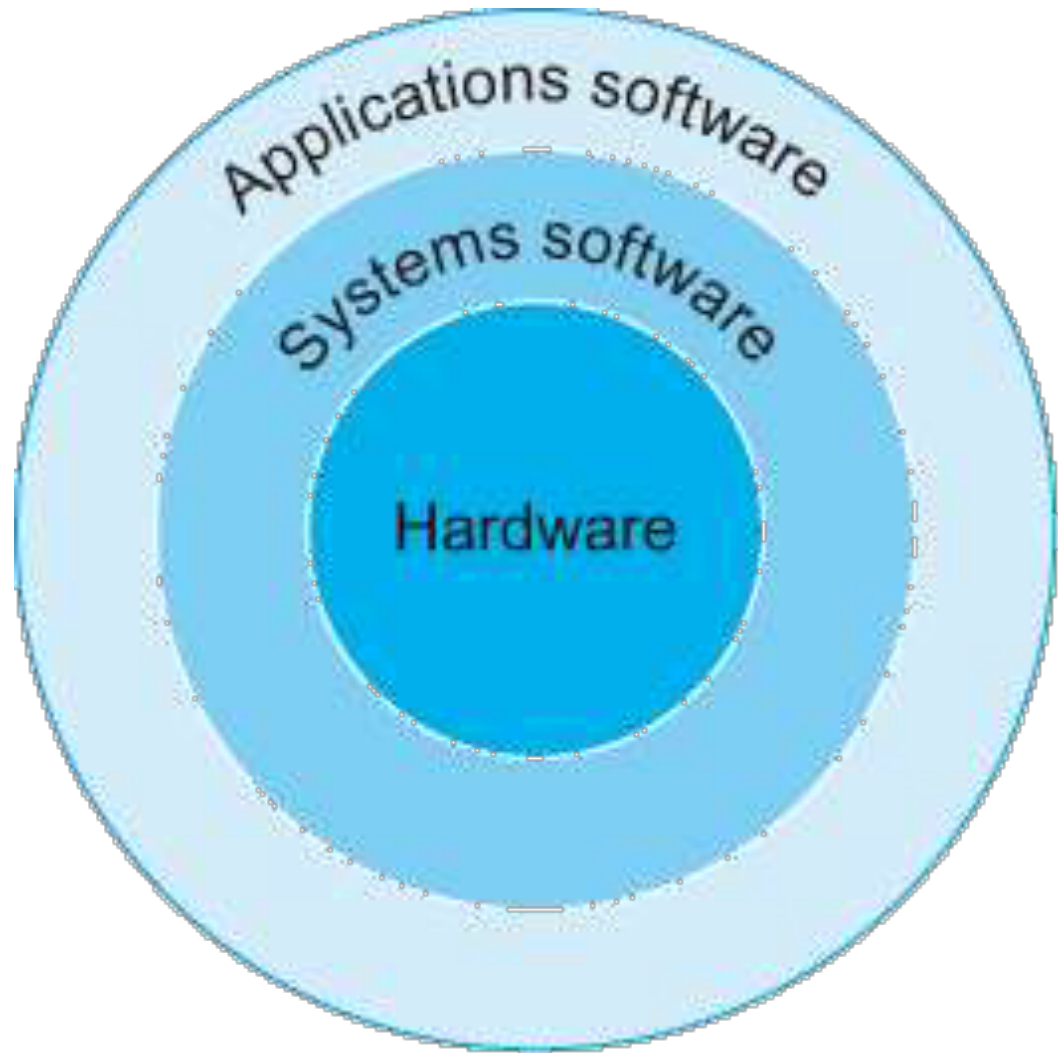
可靠性

1.3 你的程式之下(Below Your Program)

- ▣ 計算機中的硬體僅能執行極為簡單的低階指令
- ▣ 從複雜的應用落實到這些簡單硬體指令，過程中牽涉到許多層的軟體以直譯(interpret)或翻譯(translate)高階動作成為簡單計算機指令
編譯 Compile
- ▣ 系統軟體有許多種類，然而現在每一個計算機中最
重要的有兩類：
 - ① 作業系統(operating system)
 - ② 編譯器

圖1.3 以硬體為中心而
應用軟體在最外的同心圓表示的簡化硬體
軟體階層圖

在複雜的應用情形下，
也常會用到多層的應
用軟體。



由高階語言到硬體的語言

- ▣ 計算機硬體的字母僅有兩個，為二進制數元(binary digit)或位元(bit)
- ▣ 計算機接受的命令——稱之為指令(instruction)
- ▣ 組譯器(assembler)將一道以符號(symbol)表示的指令翻譯成二進形式

`add A,B` $\Rightarrow$ `1000110010100000`

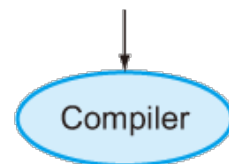
- ▣ 高階程式語言(high-level programming languages)、組合語言(assembly language)與機器語言(machine language)，圖1.4 表示該等程式與語言間的關係

圖1.4 C 程式編譯成組合語言再組譯成二進制機器語言。

雖然由高階語言轉譯成二進制機器語言在此表示為兩個步驟，有些編譯器省去中間過程而直接產出二進制機器語言。這些語言以及這個程式將在第二章中有更深入的討論

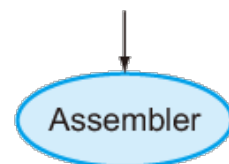
High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```



Assembly
language
program
(for MIPS)

```
swap:
    multi $2, $5, 4
    add    $2, $4, $2
    lw     $15, 0($2)
    lw     $16, 4($2)
    sw     $16, 0($2)
    sw     $15, 4($2)
    jr     $31
```



Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
10101110000100100000000000000000
101011011110001000000000000000100
00000011111000000000000000001000
```

1.4 覆蓋之下(Under the Covers)

▣ 所有計算機內的硬體都執行以下的基本功能：

① 輸入資料 (可以沒有) 

② 輸出資料 (一定要有) 

③ 處理資料

④ 儲存資料

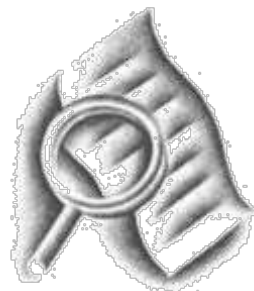
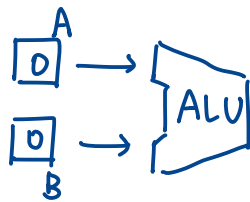
▣ 計算機的五大傳統標準要件是輸入、輸出、記憶體、數據通道(datapath)以及控制，而最後兩項有時會併稱為處理器。
CPU

圖1.5 表示出五大標準要件的計算機組織

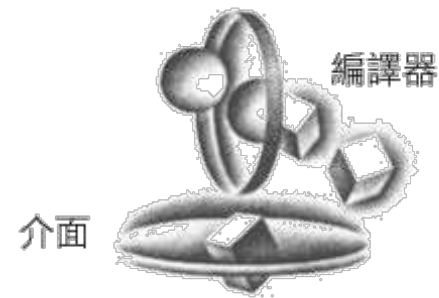
處理器由記憶體取得指令與資料。輸入將資料寫入記憶體，而輸出由記憶體讀取資料。控制送出決定數據通道、記憶體、輸入和輸出應如何運作的訊號

ALU: 算術邏輯單元

指令: 讓 Data 在不同的通道間移動實現不同的動作

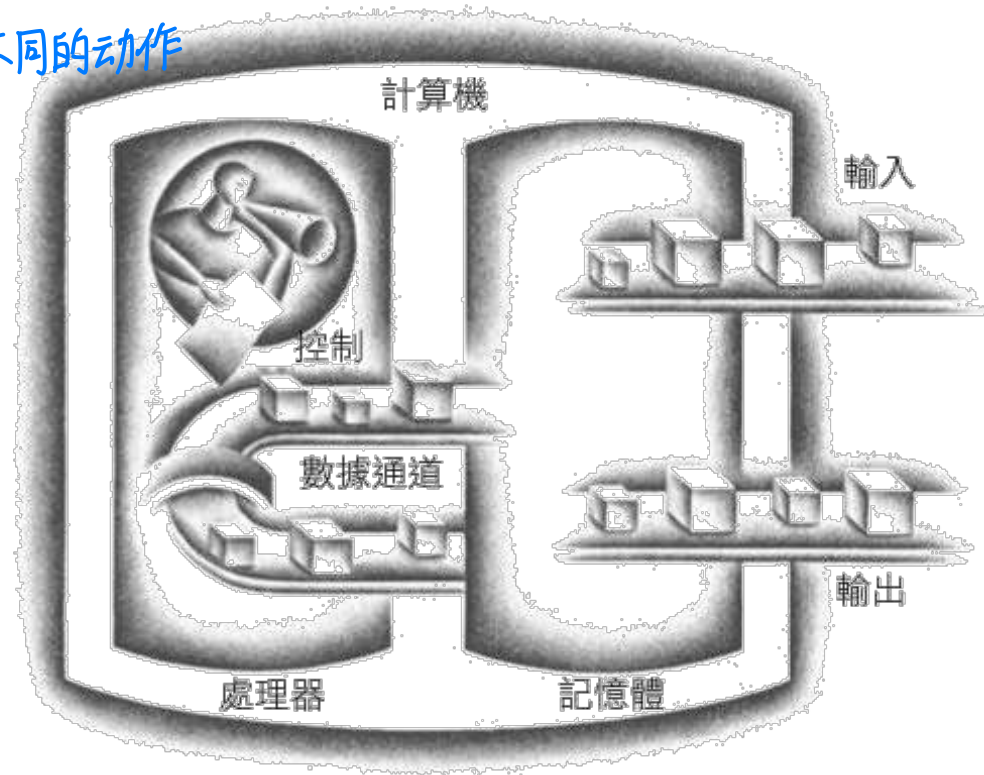


評估效能



編譯器

介面



1.5 建構處理器與記憶體的技术

- 圖1.10 表示過去不同時期所使用的技術，並對每一種技術估計其單位成本的對應效能

年度	計算機使用的技術	相對的效能/單位成本
1951	真空管(vacuum tube)	1
1965	電晶體	35
1975	積體電路	900
1995	超大型積體電路 VLSI	2,400,000
2013	極大型(Ultra large-scale)積體電路 ULSI	250,000,000,000

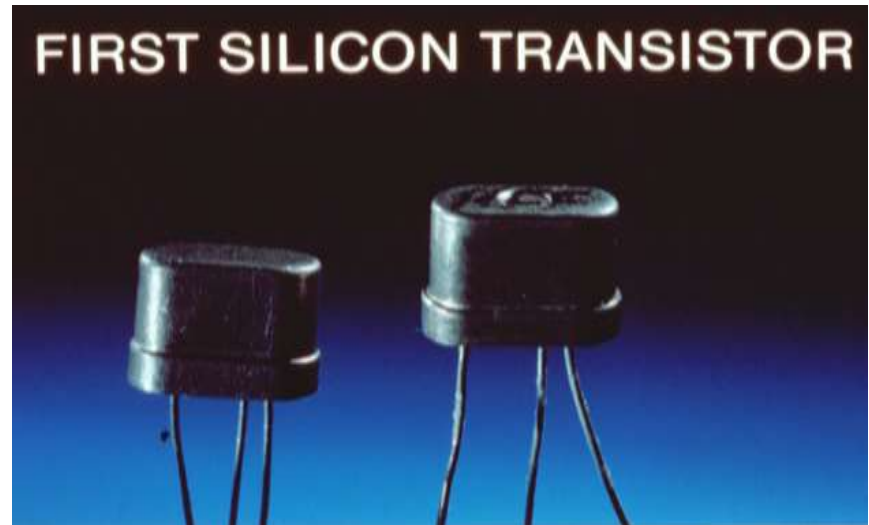
圖1.10 不同年代計算機中所使用技術的每單位成本的相對效能。

資料來源：Computer Museum，美國Boston，而2013 年的數據是由作者依外插的方式得出。Apple A16 Bionic 晶片採用4nm 製程生產，內置160 億個電晶體

真空管(vacuum tube)



電晶體



1.5 建構處理器與記憶體的技术

- 圖1.11 表示動態隨機記憶體容量自1977年以來的增長

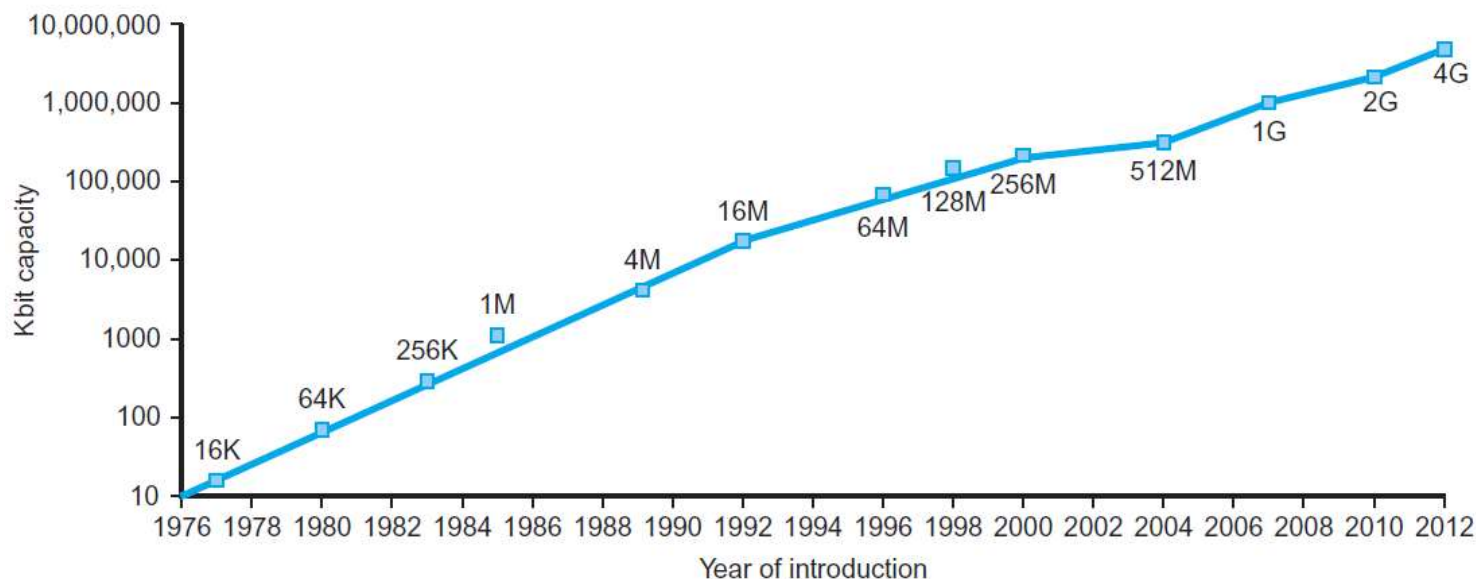


圖1.11 DRAM 晶片容量隨著時間的成長曲線

y 軸以kbits (2^{10} bits)作為單位。DRAM 業界在過去20年來幾乎每三年就將容量提高四倍，也就是每年提升60%。近年來，這個增加速度已經緩慢下來，漸漸變成約略每兩年到三年才將容量加倍

1.5 建構處理器與記憶體的技术



重要!

圖1.12 表示積體電路製程

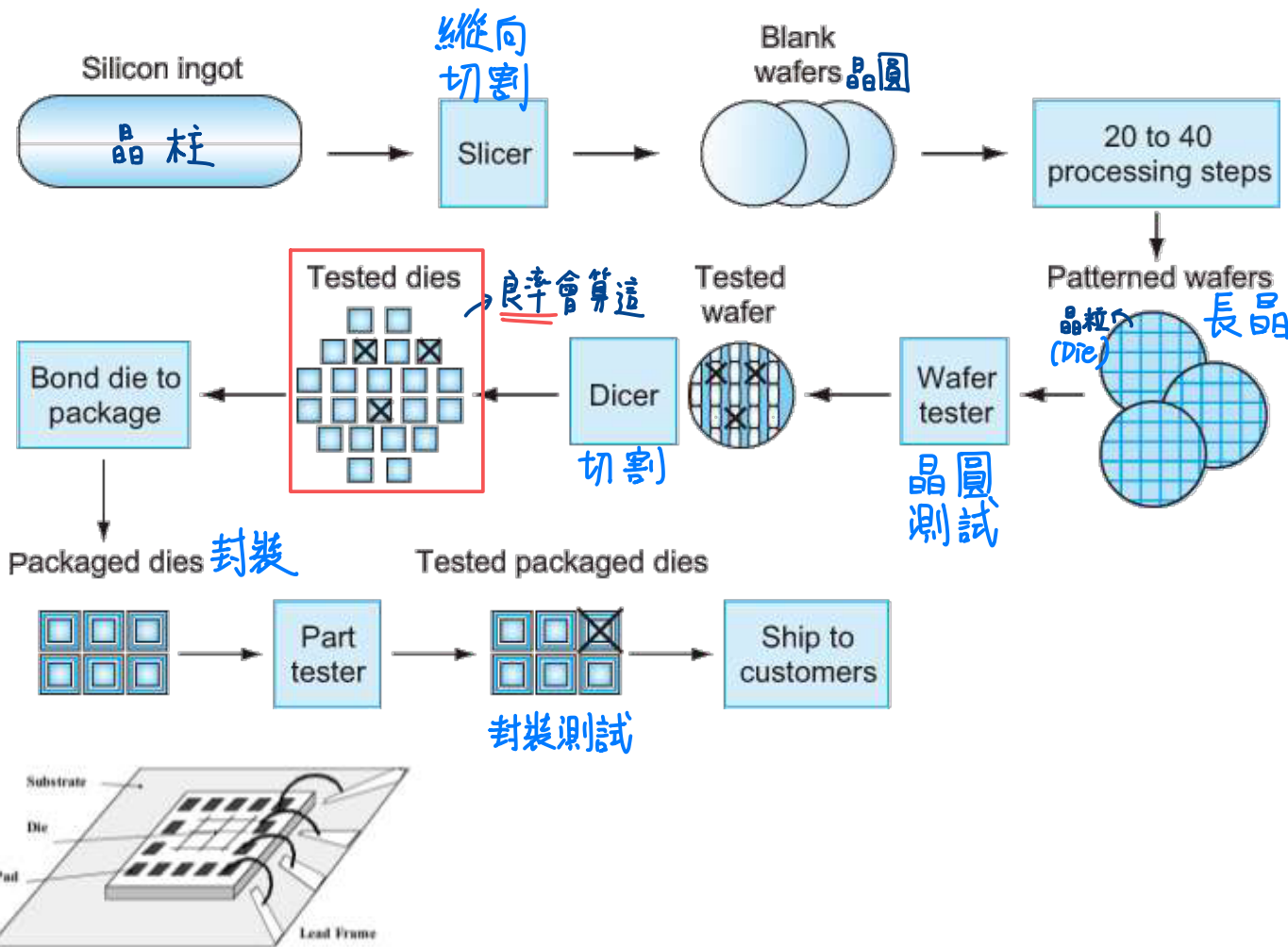
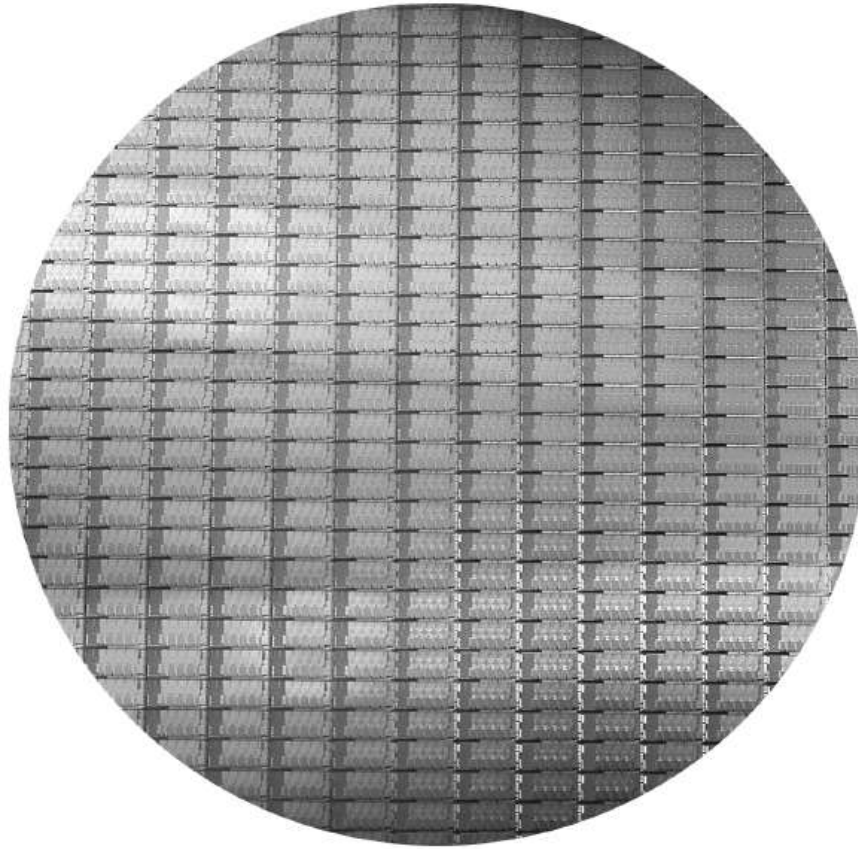


圖1.12 晶片的製造過程
空白晶圓從矽錠切片下來後，經過20到40道步驟後成為作好電路的晶圓(見圖1.13)。之後這些作好電路的晶圓經過晶圓測試機測試以產生良好部分的對照圖。之後晶圓被切割成晶粒(見圖1.9)。本圖中，一片晶圓產出20個晶粒，其中17個通過測試(x表示晶粒損壞)。本例中良好晶粒的良率(產出率，yield)是17/20或85%。這些好的晶粒接著被連結在封裝中並於出貨給顧客之前再作一次測試。在這個最後的測試中又發現一個壞掉的封裝好的零件

Intel Core i7 Wafer



- 300mm wafer(12吋), 280 chips, 32nm technology
- Each chip is 20.7 x 10.5 mm

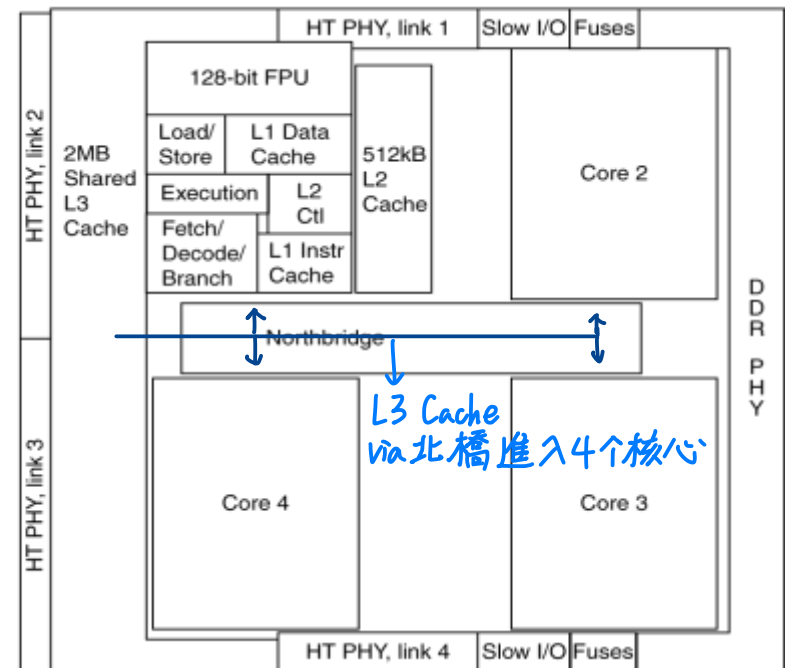
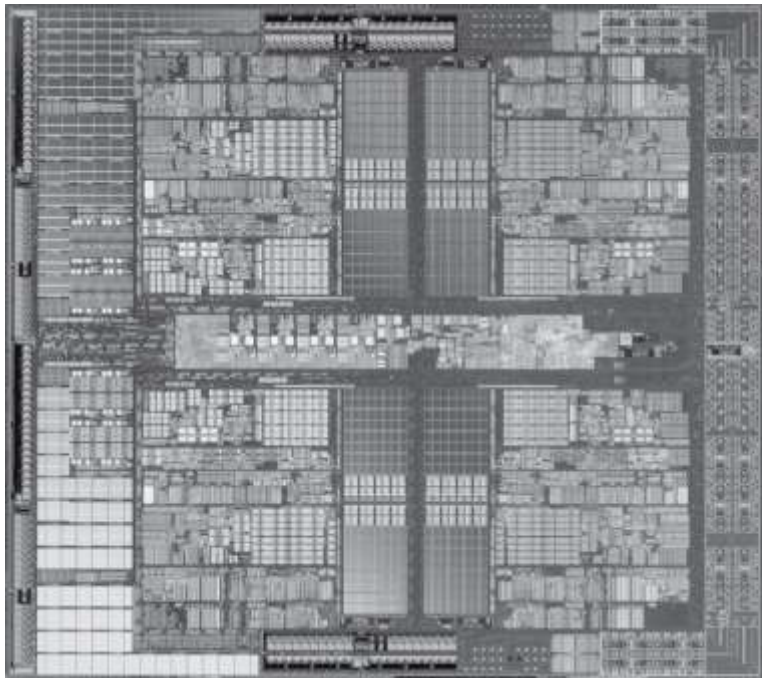
Inside the Processor

▣ Apple A5



Inside the Processor

- AMD Barcelona: 4 processor cores



Integrated Circuit Cost

晶圓成本

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

晶圓能切出幾個die 良率

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

單位面積瑕疵比例 die 的面積

單位面積瑕疵越高 ⇒ 良率越低
單位面積越大

<http://bnrg.eecs.berkeley.edu/~randy/Courses/CS252.S96/Lecture05.pdf>

■ Nonlinear relation to area and defect rate

- Wafer cost and area are fixed
- Defect rate determined by manufacturing process
- Die area determined by architecture and circuit design

"晶圓面積"由架構 & 电路设计決定

⇒ 晶圓面積 & 瑕疵率 決定良率

1.6 效能

- ▣ Response time(反應時間) — 越小越好
 - How long it takes to do a task
- ▣ Throughput(處理量) 越大越好
 - Total work done per unit time
 - e.g., tasks/transactions/... per hour
- ▣ 個人關心的是降低response time——一件工作由開始至結束的時間，亦稱為執行時間(execution time)
- ▣ 數據中心管理員則常關心處理量(throughput)或頻寬(bandwidth)在給定時間內所完成的工作量

處理量與反應時間

以下對計算機系統的改變可否提昇處理量、降低反應時間或兼得？

1. 以較快的處理器置換於計算機中 ← “使經常性的工作加快”
2. 在使用多個處理器來分別處理各個工作——例如在全球資訊網中搜尋——的系統中加入額外的處理器 → response time 不会降低, Throughput ↑

處理量與反應時間

- ❑ 降低反應時間幾乎永遠可提升處理量。因此在情況1中，反應時間及處理量均獲改善。在情況2中，沒有任何工作可更快完成，故僅處理量有提昇
- ❑ 當情況2中，處理量需求大增時，可能造成系統將工作需求貯存起來。在這種情況下，由於增加處理量可降低工作需求在貯列中的等待時間，故可同時改善反應時間。

Relative Performance

效能 = 執行時間的倒數

- ▣ Define Performance = $1/\text{Execution Time}$
- ▣ “X is n time faster than Y”

$$\begin{aligned} \text{Performance}_X / \text{Performance}_Y \\ = \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

- **Example: time taken to run a program**
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5$
 - So A is 1.5 times faster than B

Measuring Execution Time

▣ Elapsed time (經過時間)

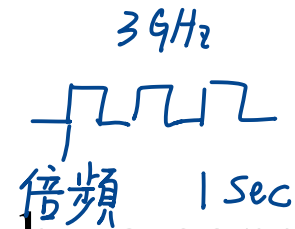
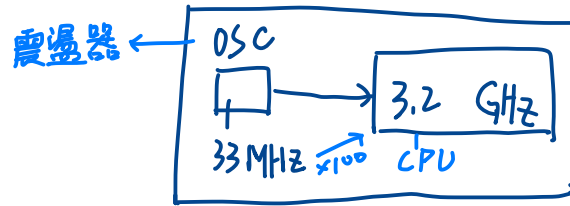
- Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
CPU 處理時間 OS 波霸的時間 (Time Sharing) 待機時間
- Determines system performance

▣ CPU time → CPU時間: CPU單純服務你的時間(扣掉I/O, Idle之類的)

- Time spent processing a given job
 - Discounts I/O time, other jobs' shares
- Comprises **user CPU time** and **system CPU time**
CPU 服務你的程式 你的程式呼叫的 System Call 所耗時間

▣ Different programs are affected differently by CPU and system performance 不同的程式会被不同的CPU和系統效能所影響。

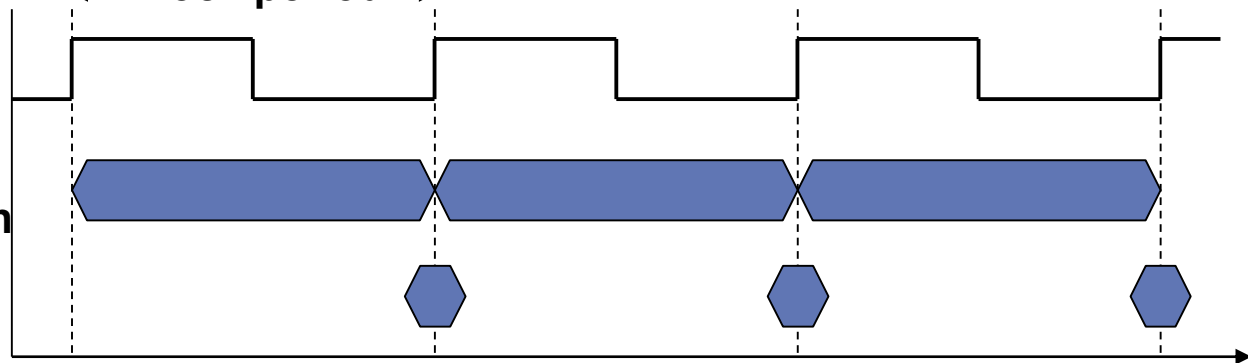
CPU Clocking



Operation of digital hardware governed by a constant-rate clock

一个Clock, 一核做一个指令

$$\frac{1}{250 \text{ (ps)}} = \frac{1}{250 \times 10^{-12}} = 4 \times 10^9 \text{ Hz}$$



■ **Clock period: duration of a clock cycle**

■ e.g., $250 \text{ ps} = 0.25 \text{ ns} = 250 \times 10^{-12} \text{ s}$

■ **Clock frequency (rate): cycles per second**

■ e.g., $4.0 \text{ GHz} = 4000 \text{ MHz} = 4.0 \times 10^9 \text{ Hz}$

CPU Time ☆

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

周期
(= CPU 頻率倒數)

- Performance improved by 改善效能(降低CPU Time)
 - Reducing number of clock cycles 減少 clock cycles 數
 - Increasing clock rate 增加 CPU 頻率
 - Hardware designer must often trade off clock rate against cycle count
- CPU 要跑得快 ⇒ 指令就不能太複雜 ⇒ 硬體設計得在 Clock Rate & 指令數目
做一個妥協

CPU Time Example *Practice*

A 執行了: $2 \times 10^9 \times 10$ 个 CPU cycles

■ Computer A: 2GHz clock, 10s CPU time

■ Designing Computer B
(1sec 2×10^9)

■ Aim for 6s CPU time

■ Can do faster clock, but causes $1.2 \times$ clock cycles

■ How fast must Computer B clock be?
問 Frequency

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B}$$

$$= \frac{1.2 \times \text{Clock Cycles}_A}{\text{CPU Time}_B}$$

$$\text{Clock Cycles}_A = 10 \times 2 \times 10^9 = 2 \times 10^{10}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 2 \times 10^{10}}{6} = 4 \times 10^9 = 4\text{GHz}$$

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6\text{s}}$$

$$\text{Clock Cycles}_A = \text{CPU Time}_A \times \text{Clock Rate}_A$$

$$= 10\text{s} \times 2\text{GHz} = 20 \times 10^9$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6\text{s}} = \frac{24 \times 10^9}{6\text{s}} = 4\text{GHz}$$

Instruction Count and CPI(Cycles Per Instruction)

Practice

每个 Command 要几个 Cycle 才能做完?

— 越低越好

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$$

指令數目 CPI

$$\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}} \times \text{Clock Cycle Time}$$

Clock Cycles

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

■ Instruction Count for a program 產生指令數目?

- Determined by program, ISA(Instruction Set Architecture) and compiler 指令集架構

■ Average cycles per instruction 每個指令平均 Cycle 數由硬體決定

- Determined by CPU hardware
- If different instructions have different CPI
 - Average CPI affected by instruction mix

CPI Example *Practice*

一個程式在不同的電腦執行

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same Instruction count
- Which is faster, and by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \quad \leftarrow \text{A is faster...}\end{aligned}$$

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2 \quad \leftarrow \text{...by this much}$$

A 比 B 快了 1.2 倍

CPI in More Detail

- If different instruction classes take different numbers of cycles

n種不同指令

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

■ Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency

CPI Example

相同程式用不同編譯器，在相同電腦執行

- Alternative compiled code sequences using instructions in classes A, B, C

- IC : Instruction count (指令數目)

different Compiler

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5

- Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$

- Avg. CPI = $10/5 = 2.0$

- Sequence 2: IC = 6

- Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$

- Avg. CPI = $9/6 = 1.5$

Performance Summary

The BIG Picture

CPU Time 怎麼算?

指令數

CPI

週期

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

一個程式有多少指令

一個指令有多少Cycle?

每個Cycle
所需的時間

Performance depends on

決定效能4關鍵

演算法 ■ Algorithm: affects IC, possibly CPI

程式語言 ■ Programming language: affects IC, CPI

編譯器 ■ Compiler: affects IC, CPI

指令集架構 ■ Instruction set architecture: affects IC, CPI, T_c

瞭解程式效能

- 一個程式的效能與演算法、語言、編譯器、架構以及實際的硬體有關

硬體或軟體要素	影響什麼？	如何影響？
演算法	指令數，可能 CPI	演算法決定執行的高階程式指令數目，也因此決定執行的處理器指令數目。演算法對較快或較慢指令的偏好也可能影響 CPI。舉例而言，若演算法使用較多的浮點運算，則可能導致更高的 CPI。
程式語言	指令數，CPI	程式語言當然影響指令數，因為其敘述句將被轉譯成處理器指令，因而決定指令數。語言的特性也可能影響 CPI；舉例而言，一個大量支援資料抽象化(data abstraction)的語言(例如 Java)將需要間接呼叫的功能，如此將使用 CPI 較高的指令。
編譯器	指令數，CPI	由於編譯器決定將高階語言指令轉換成計算機的指令，其效率會影響指令數及每指令平均週期數。編譯器的角色可以相當複雜，並以複雜的方式影響 CPI。
指令集架構	指令數，時脈速率，CPI	由於指令集架構影響一個功能所需的指令、每道指令的時脈數以及處理器整體時脈速率，因此其影響中央處理器效能相關的所有方面。

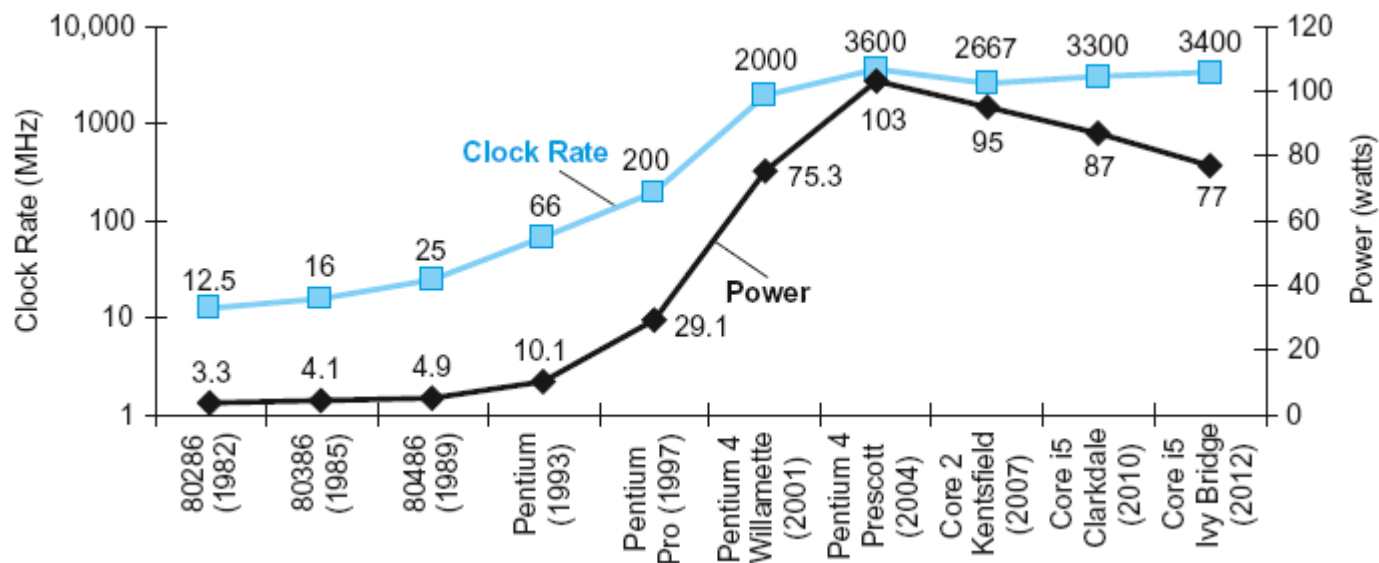
- 有些處理器每時脈週期擷取及執行多道指令
 - 有些設計者倒轉CPI以表為IPC，或每時脈指令數。
IPC、CPI互為倒數
Instruction Per Clock
每Clock能產生幾個指令
 - 若某處理器平均每時脈執行二道指令，則其具有 $IPC=2$ 亦即 $CPI=0.5$
- 為了省能或是短暫地加強效能，現在的許多處理器可以變化它們的時脈速率，因此我們對一個程式會需要知道其平均的時脈速率

1.7 功耗障壁(The power wall)

- ❑ 功耗考驗冷卻能力，一個晶片的能量消耗與其冷卻運作能力已經成為確保它能夠正常工作的基本要求
 - 電送進去，熱排出來
- ❑ 對能源而言焦耳這個能量單位比瓦數的耗能率更為恰當
 - 所消耗的電能，稱為電功率，其單位為瓦特（watt，簡寫為W），簡稱瓦。◎如果有1安培的電流通過1伏特的電位差，則每秒內所獲得或消耗的能量為1焦耳，或者說電功率為1瓦特，即「1瓦特=1焦耳秒」，或「1W=1J/s」。
IC基本上只會在轉態(0↔1)時產生功耗
- ❑ 每個電晶體 0/1 轉換一次的動態能耗是  省电模式: 拉大Clock, 減少轉態
$$\text{能耗} \propto \frac{1}{2} \times \text{電容性負載} \times \text{電壓}^2$$
- ❑ 每個電晶體所需功耗就是
$$\text{能耗} \propto \frac{1}{2} \times \text{電容性負載} \times \text{電壓}^2 \times \text{切換頻率}$$

Power Trends

功耗 & Clock 的關係



■ In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

電 容 器 負 載

電 壓²

頻 率

×30

5V → 1V

×1000

功耗障壁

- ▣ 一般電壓在每個新世代中下降大約15%
 - 20 年來，電壓已從5V 降到1V
- ▣ 今日的問題是更加降低電壓將會使得電晶體漏電太多
 - 甚至於今天電耗中已有40% 來自於漏電
- ▣ 兩個理由使功耗成為積體電路設計的難題
 - ① 電源必須由外部引入且分送至晶片各處 電怎麼進去？
 - ② 功耗以熱的形式消散然後必須被移除 熱怎麼出來？

1.8 巨變：由單處理器轉移至多處理器

- ▣ 圖1.17 顯示桌上型微處理器在程式反應時間上隨著年代的改善
 - 2002 年起改善速率減緩
- ▣ 2006 年時所有桌上型及伺服器公司都推出每一晶片具有多個處理器的微處理器
 - 其所帶來在處理量上的好處更甚於反應時間，稱處理器為內核
 - 稱這種微處理器為多核微處理器



44

1.8 巨變：由單處理器轉移至多處理器

- ▣ 程式師如欲獲致重大的時間改善，則需要重新編程以善用多個處理器
- ▣ 為什麼撰寫明確的平行程式這麼難？
 - 第一個理由是追求平行效能的程式寫作困難度增加
 - 第二個理由是程式師必須分割一個應用以使每個處理器同時可以擁有大約相同的工作量，以及排程和協調所造成的額外花費不致於抵銷掉平行性帶來的潛在效能好處
 - 均勻地平衡負載(balance the load)
 - 降低通訊與同步的額外花費(reduce communication and synchronization overhead)

1.9 實例：測試Intel Core i7

SPEC 中央處理器測試程式

- 大多數使用者依賴已知方法來測量受測計算機的效能，並期待該方法足以反映某一計算機對他們的工作負載表現有多好
 - e.g. 整數運算、浮點數運算...
 - 通常是以一組測試程式(benchmarks)——特別挑選來量測效能的程式——來評量計算機
 - 測試程式集組成一個使用者希望足以預測真實工作負載效能表現的工作負載

SPEC 中央處理器測試程式

- SPEC(系統效能評估組織，System Performance Evaluation Cooperative)是許多計算機業者為了創造現代計算機系統各種標準測試程式集而建立及支持的組織
 - 1989 年SPEC 首次提出SPEC89
 - 最新的一代稱為SPEC CPU2006
 - 包含了一組12 個的整數測試程式(CINT2006)以及17 個的浮點測試程式(CFP2006)

SPEC 中央處理器測試程式

▣ SPEC CPU2006

- Elapsed time to execute a selection of programs
 - Negligible(忽略) I/O, so focuses on CPU performance
- Normalize^{正規化} relative to reference machine
 - *SPECratio*
將測試資料正規化, 得出一個相對於參考機器的比值
- Summarize as geometric mean of performance ratios
 - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i} \quad (\text{幾何平均數})$$

CINT2006 for Intel Core i7 920

Description	Name	Instruction Count x 10 ⁹	CPI	Clock cycle time (seconds x 10 ⁻⁹)	Execution Time (seconds)	Reference Time (seconds)	SPECratio
Interpreted string processing	perl	2252	0.60	0.376	508	9770	19.2
Block-sorting compression	bzip2	2390	0.70	0.376	629	9650	15.4
GNU C compiler	gcc	794	1.20	0.376	358	8050	22.5
Combinatorial optimization	mcf	221	2.66	0.376	221	9120	41.2
Go game (AI)	go	1274	1.10	0.376	527	10490	19.9
Search gene sequence	hmmer	2616	0.60	0.376	590	9330	15.8
Chess game (AI)	sjeng	1948	0.80	0.376	586	12100	20.7
Quantum computer simulation	libquantum	659	0.44	0.376	109	20720	190.0
Video compression	h264avc	3793	0.50	0.376	713	22130	31.0
Discrete event simulation library	omnetpp	367	2.10	0.376	290	6250	21.5
Games/path finding	astar	1250	1.00	0.376	470	7020	14.9
XML parsing	xalancbmk	1045	0.70	0.376	275	6900	25.1
Geometric mean	–	–	–	–	–	–	25.7

SPEC 功耗測試程式

■ SPEC 測試功耗的程式

■ SPECpower 測出伺服器在一段時間裡以10% 為區間的負載情況下的功耗

- 為了簡化計算機的市場宣傳，SPEC 將這些數字整理成一個單一數字，稱為 **Overall ssj_ops per watt**
- SPECpower_ssj2008 is basically a measure of **ssj_ops/watt** (server side Java operations per second per watt)
- Performance: ssj_ops/sec
- Power: Watts (Joules/sec)

~ 1瓦特可以跑多个JAVA的 operation

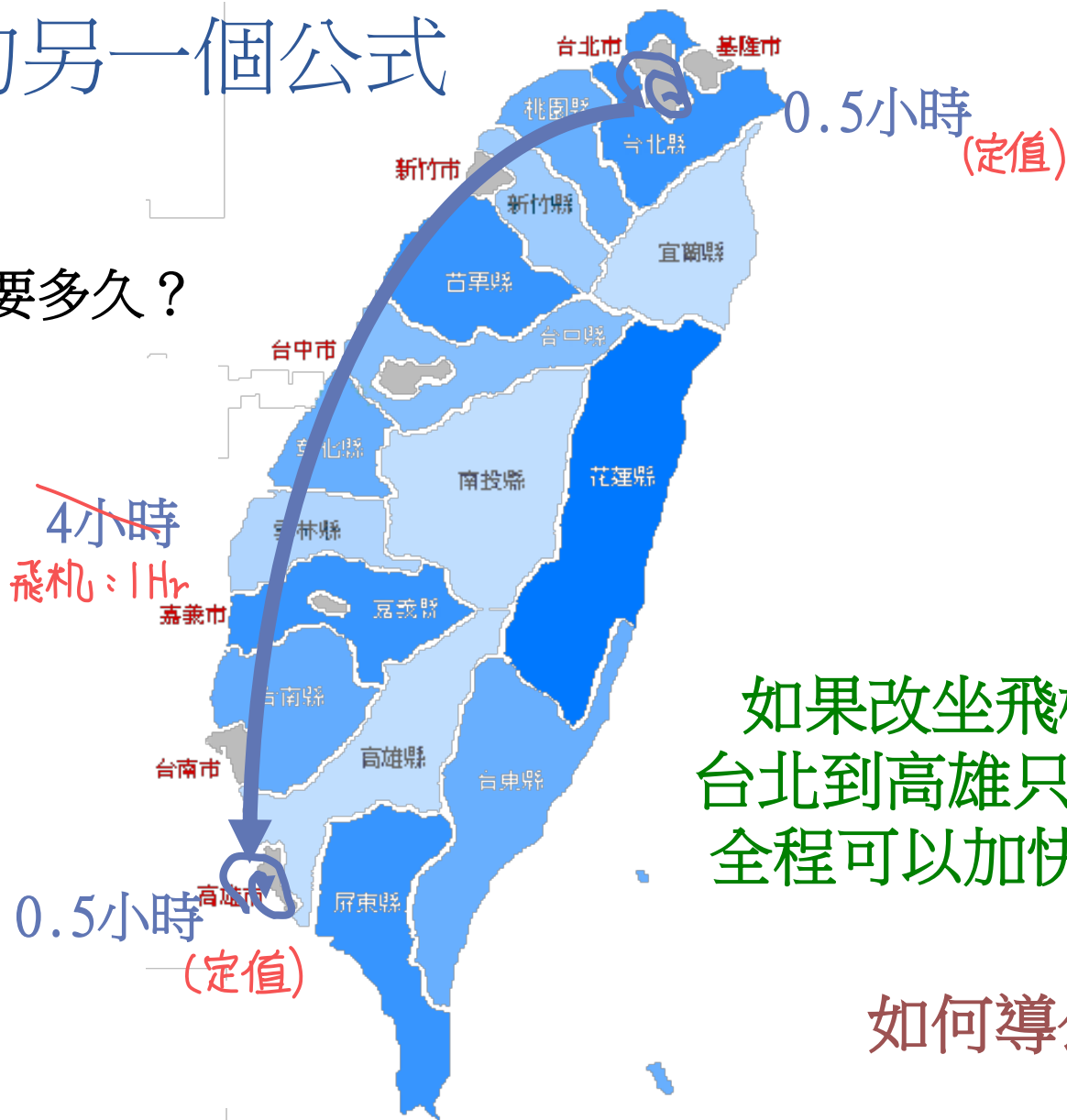
$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) / \left(\sum_{i=0}^{10} \text{power}_i \right)$$

SPECpower_ssj2008 for Xeon X5650

Target Load %	Performance (ssj_ops)	Average Power (Watts)
100%	865,618	258
90%	786,688	242
80%	698,051	224
70%	607,826	204
60%	521,391	185
50%	436,757	170
40%	345,919	157
30%	262,071	146
20%	176,061	135
10%	86,784	121
0%	0	80
Overall Sum	4,787,166	1,922
$\Sigma \text{ssj_ops} / \Sigma \text{power} =$		2,490

有關效能的另一個公式

從台北到高雄要多久？



由台北到高雄

- ▣ 不能enhance的部份為在市區的時間: $0.5 + 0.5 = 1$ 小時
- ▣ 可以enhance的部份為在高速公路上的4小時
- ▣ 現在改用飛機, 可以enhance的部份縮短為1小時

▣

$$\text{speedup} = \frac{\text{走高速公路所需時間} \quad 4 + 1}{\text{坐飛機所需時間} \quad 1 + 1} = \frac{5}{2} = 2.5$$

1.10 謬誤(fallacies)與陷阱(pitfalls)

- ▣ 陷阱：期望計算機某一方面的改善可以提昇整體效能到等同於該改善的幅度 (e.g. I/O)

- **Amdahl's** 定律

改善後的執行時間

$$= \frac{\text{受改善影響的執行時間}}{\text{改善的幅度}} + \text{不受影響的執行時間}$$

- ▣ 回收遞降定律(The law of diminishing returns)
(會有遞減效應)

1.10 謬誤與陷阱

- ▣ 謬誤：低利用率的計算機幾乎不耗電
- ▣ 謬誤：為了效能而設計以及為了能量效益而設計是不相關的目標
- ▣ 陷阱：使用效能計算式的一部分作為效能衡量標準
 - **MIPS** (百萬指令每秒，million instructions per second)

$$\text{MIPS} = \frac{\text{指令數}}{\text{執行時間} \times 10^6}$$

- MIPS 的好處是其易懂，較快的計算機有較大的 MIPS，符合直覺

Pitfall: MIPS as a Performance Metric

■ MIPS: Millions of Instructions Per Second

- Doesn't account for
 - ① 指令集架構需相同
 - Differences in ISAs between computers
 - ② 指令複雜度需相同
 - Differences in complexity between instructions
- ①. ② 都滿足, MIPS才能比較

$$\begin{aligned}\text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}\end{aligned}$$

Pitfall: MIPS as a Performance Metric

- 有三個問題：

① MIPS說明指令執行速率然而未考慮指令的能力
 $\Rightarrow$ Now: RISC (精簡指令集) $\leftarrow$ MIPS $\uparrow$
以前: CISC (複雜指令集) $\leftarrow$ MIPS $\downarrow$

② 即使在同一計算機上，MIPS也隨程式而異

③ 程式可改寫成執行更多指令，然而各指令執行得更快！